One P-Bit per Parameter
Supplementary Methods, Diagnostics, and Evidence

Omar Ramadan
Independent Research

November 6, 2026
Public-access revision: November 8, 2026

Contents

1 Scope and reading guide **2**

2 Checkpoint, task, and numerical contracts **2**

3 Finite encoder and reconstruction details **2**
3.1 Grouping and constant groups . . . 2
3.2 Independent counts and random keys . . . 3
3.3 API example . . . 3
3.4 Information bound . . . 3

4 Inference statistics and complete cohorts **3**

5 Exact sampling-allocation procedure **4**

6 Cumulative and causal diagnostics **4**
6.1 GPT-2 cumulative substitutions . . . 4
6.2 LFM3 intervention coverage . . . 5
6.3 Signal scale and neuron structure . . . 5

7 Covariance estimator and interpretation **7**

8 Additional controls and historical outcomes **7**
8.1 Matched Gaussian noise . . . 7
8.2 Correlation, drift, and calibration . . . 7
8.3 Earlier experiments and failures . . . 8

9 Resource ledger and storage boundary **9**

10 Evidence, reproduction, and outstanding work **9**

1 Scope and reading guide

This supplement accompanies the main manuscript. It records exact execution conventions, diagnostic controls, resource accounting, and the relationship between historical and accepted experiments. The original research plan is preserved as `master_design.tex`; it is a plan, not evidence that every proposed phase was completed. The research repository preserves completed, failed, interrupted, and unexecuted protocols separately. Only completed and audited observations support the numerical claims below.

There are three evidence levels: ideal independent-count simulation at full-model scale; direct official THRML sampling at operator scale; and physical devices. The third level is not part of this study. The term K1 counts one logical programmable stochastic element per scalar. It does not mean one bit of total storage, a physical allocation on a chip, or deterministic one-bit neural arithmetic.

2 Checkpoint, task, and numerical contracts

The evaluated checkpoints are GPT-2, `distilbert/distilbert-base-uncased-finetuned-sst-2-english`, and `LiquidAI/LFM3-1.7B`. LFM3 is pinned to revision `ac2586a34f1a1c03329dbdcdefabd8b6478ddaa7`. Its weight file SHA-256 is

`135922caef64e81e52af88ede8b7c3ddaa9d6da220a9737fb3065275dbfb4330`.

All remaining checkpoint and input hashes are in the bundled protocol files and research archive. Checkpoint weights are not redistributed in this Git repository.

Each R1 condition reconstructs one complete set of model weights and reuses it across all examples. Thus sampling seeds, not individual tokens, are the independent Monte Carlo units. GPT-2 scores 283,225 next-token targets over 586 windows and 62 articles. LFM3 scores 294,442 targets over 607 windows and 62 articles using its own tokenizer. DistilBERT scores all 25,000 balanced IMDb test reviews, truncated/padded to length 256, in batches of 32. The different tokenizer counts are intentional. These language-model evaluations are teacher-forced NLL measurements, not autoregressive generation benchmarks.

The original weights are rounded to BF16 before the stochastic comparison. Main model evaluations use BF16 neural computation. Diagnostic CPU experiments explicitly identified as such instead use FP32 arithmetic on BF16-rounded original and reconstructed weights. Eager attention and deterministic Torch settings are recorded in the source. LFM3 uses its original full-sequence output head and no KV cache. The custom fast convolution path is disabled in the recorded offload runs; this implementation choice is shared with their controls.

Native THRML is pinned to commit `2d54cd8404df85bb65703d7e8b4e84e48c1b72a9`, reported as version 0.1.4, with JAX 0.12.0.1 in the tested environment. Full-model runs use Torch 2.14.1+cu130 and NumPy 2.5.3. The optional SDK dependency specification records the complete tested pins. Runtime records are in the archive, rather than inferred from a package version alone.

3 Finite encoder and reconstruction details

3.1 Grouping and constant groups

The encoder accepts finite FP32-representable weights. A matrix can be flattened in storage order or transposed before flattening; the inverse permutation is applied after reconstruction. Groups are contiguous after that permutation, including a possibly shorter final group. A group stores its FP32

minimum and FP32 span, and one 12-bit command per scalar in a uint16 array. A zero span means the reconstructed group is exactly constant irrespective of the sampled state.

The probability field is clipped to $[-4, 4]$ and rounded to 4,096 equally spaced command levels. Decoding evaluates the field in FP32 and the logistic probability in FP64. This exact order is important for hash replay. The supported group sizes in the SDK range from one through 65,536; the reported model configurations use 7, 16, or 128 as specified, not a per-parameter range that hides an extra full-precision copy.

3.2 Independent counts and random keys

Reconstruction calls NumPy's binomial generator once per scalar instead of emitting T explicit Bernoulli bits. Each canonical tensor uses a random stream initialized from the experimental seed and tensor index. Groups are processed in deterministic chunks aligned to their boundaries. Counts use uint16 through $T = 65{,}535$, and uint32 above that threshold. A SHA-256 digest over counts and a digest over reconstructed BF16 values are preserved in per-tensor records.

A binomial draw is an exact distributional shortcut only for independent identically distributed observations. It does not model temporal or cross-device correlation. The native and nonideal experiments are separate. Python random-number generation time also cannot be interpreted as a physical device's sampling time.

3.3 API example

The standalone package can reconstruct a finite-control program without model checkpoints:

```
import numpy as np
from pbit1 import FinitePBitProgram

w = np.array([[0.1, -0.2], [0.6, 0.3]], dtype=np.float32)
program = FinitePBitProgram.encode(w, group_size=2)
sample = program.reconstruct(32768, seed=7)
print(sample.weights)
print(program.storage())
```

The return value is FP32. The research pipeline applies its explicit BF16 conversion afterward. The SDK's storage report counts array payload, not Python-object overhead, a reconstructed matrix, a retained reference model, or external hardware.

3.4 Information bound

For T independent Bernoulli observations with interior probability p, the Fisher information is $\mathcal{I}(p) = T/[p(1-p)]$. The sample mean attains the Cramér–Rao variance bound $p(1-p)/T$ for unbiased estimation of p. Under a fixed affine range with span d, weight variance is $d^2 p(1-p)/T$. This quantifies an estimation limit; it does not prove that a neural task needs that much parameterwise accuracy. Finite probability controls add a separate bias term that does not vanish with T.

4 Inference statistics and complete cohorts

Table S1 preserves all five selected-policy language-model perplexities. Their geometric mean corresponds to exponentiating average seed NLL; their arithmetic mean is not the reported main PPL. Seed IDs are experimental identifiers, not calendar dates.

Table S1: Complete language-model selected-policy cohorts.

GPT-2 seed	PPL	LFM3 seed	PPL
2026110801	25.60020	2026111901	18.98507
2026110802	25.54822	2026111902	18.59381
2026110803	25.58146	2026111903	19.17330
2026110804	25.92167	2026111904	19.01270
2026110805	25.69791	2026111905	19.13664

All primary intervals use the five seed-level deltas and a t_4 critical value of 2.776445. The final LFM3 joint seed/article bootstrap uses 5,000 replicates and produces [0.99188, 1.01065]. Recomputing statistics on saved target losses verifies coverage and scoring, but it does not establish independence of tokens or newly unseen data. Both are explicit limitations.

5 Exact sampling-allocation procedure

The gradient proxy averages $\sum_i (\partial \mathcal{L}/\partial w_i)^2 v_i$ over eight short training windows, where v_i is the single-sample variance. Candidate one-dimensional ranges use group sizes 16, 32, 64, and 128. Matrix candidates retain group size 128 with the two flattening orientations. A second group of eight calibration windows evaluates proposals before freezing the complete GPU test.

The executed discrete allocation starts each tensor at $T_i = 1{,}024$. Let S_i be its selected gradient-weighted variance coefficient and N_i its size. Doubling T_i reduces proxy variance by $S_i/(2T_i)$ at an added cost of $N_i T_i$ observations. The greedy ratio is therefore

$$\frac{S_i}{2N_i T_i^2}. \tag{S1}$$

The algorithm repeatedly doubles the eligible tensor with the largest ratio until it meets the recorded proxy target or reaches the cap $T_i \leq 32{,}768$. The final accepted policy is `gradient_proxy_directions_vectorG16`. This is a fixed per-tensor schedule selected offline, not confidence-based early stopping for individual examples. The original plan's adaptive stopping and probability-aware fine-tuning remain separate possible extensions.

The full evaluation uses five fresh seeds, 2026111401 through 2026111405. Its frozen acceptance conditions require complete target coverage, all learned parameters K1, maximum sample count 32,768, nominal cost at most 60% of uniform, and upper PPL-ratio interval at most 1.02. The observed cost is 58.324%, with upper interval 1.01660. The BF16 reference arrays match the earlier complete baseline.

6 Cumulative and causal diagnostics

6.1 GPT-2 cumulative substitutions

The 41-condition CPU experiment consists of one baseline and eight configurations for each of five seeds: prefix and suffix groups of 3, 6, or 9 blocks, all twelve blocks, and all parameters. The arithmetic is FP32 on BF16-rounded weights, using 1,024 training targets. Nonblock parameters remain deterministic in the prefix/suffix and all-block controls; only the all-parameter condition converts every learned scalar.

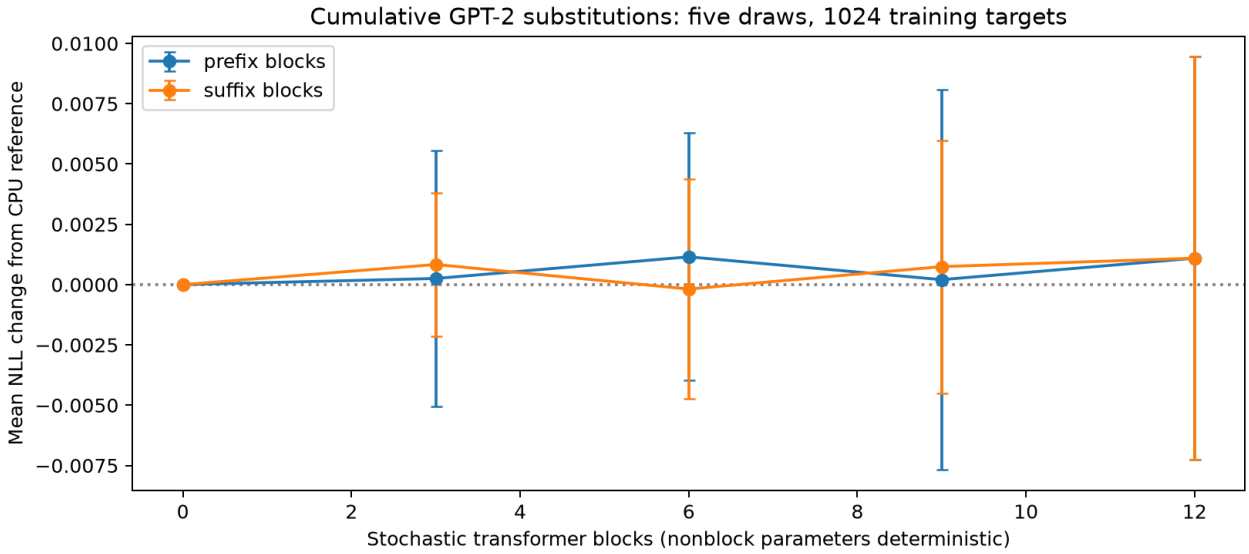


Figure S1: GPT-2 cumulative conversion on five training draws. Intervals overlap zero for the reported body-only curves, and loss is not monotonic in converted block count. These results do not prove a monotonic or additive layer-sensitivity model.

All masks, scores, and saved input contracts were audited; 148 SDK count hashes and six neural rows replayed. Small differences in loss must be interpreted at this limited sample size, rather than as a ranking of universally dispensable layers.

6.2 LFM3 intervention coverage

The 55-condition LFM3 causal study uses eight training windows of 129 input tokens, yielding 1,024 scored targets. The fixed draw uses seed 2026111801. One baseline, one all-K1 condition, 24 isolated substitutions, 24 restorations, and five family restorations are recorded. The isolated interventions leave every earlier block unchanged. Six independently reconstructed neural training rows and all 286 count records establish the replay contract before the intervention analysis.

6.3 Signal scale and neuron structure

For each LFM3 block, the baseline activation study uses 8,224 positions and 2,048 channels. The activation energy participation ratio is $(\sum_j e_j)^2 / \sum_j e_j^2$, where e_j is the channel second moment. Ratios range approximately from 9 to 654 across blocks. This measures concentration of activation energy, not the number of removable neurons.

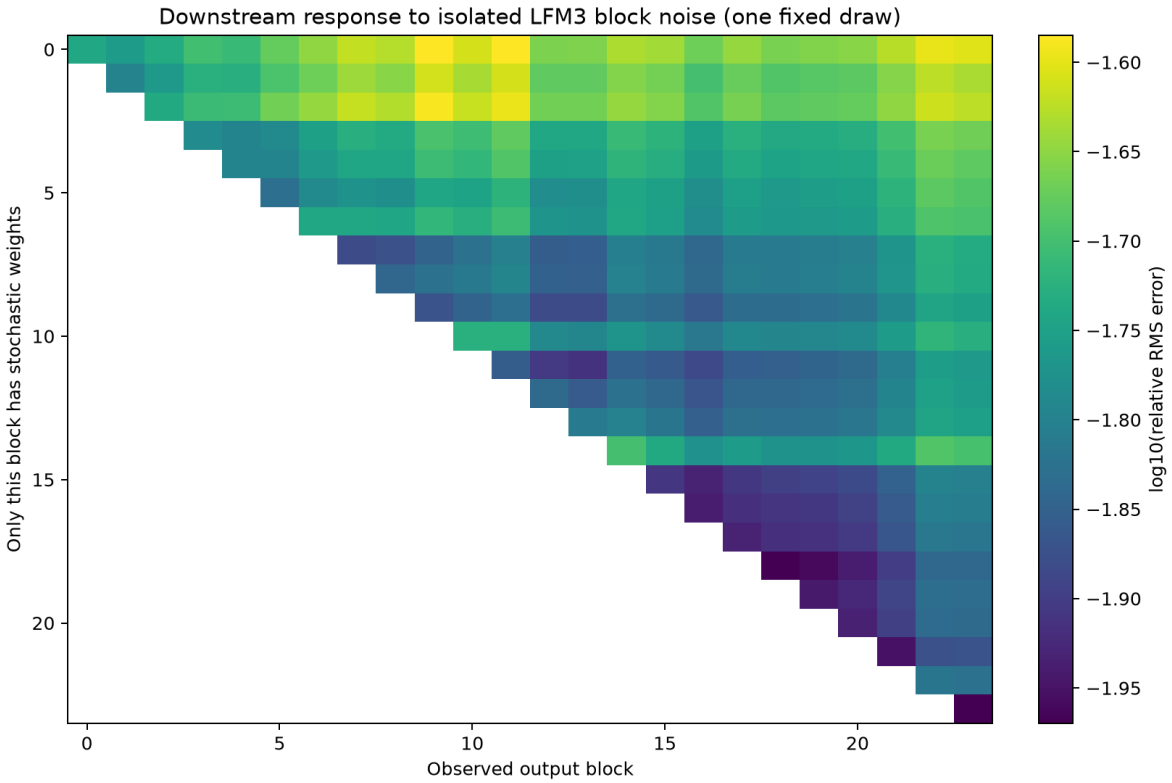


Figure S2: Downstream LFM3 responses to isolated substitutions. Earlier blocks are exactly unaffected. Absolute responses depend on the operating point and include subsequent nonlinear computation; this matrix is not a global stability guarantee.

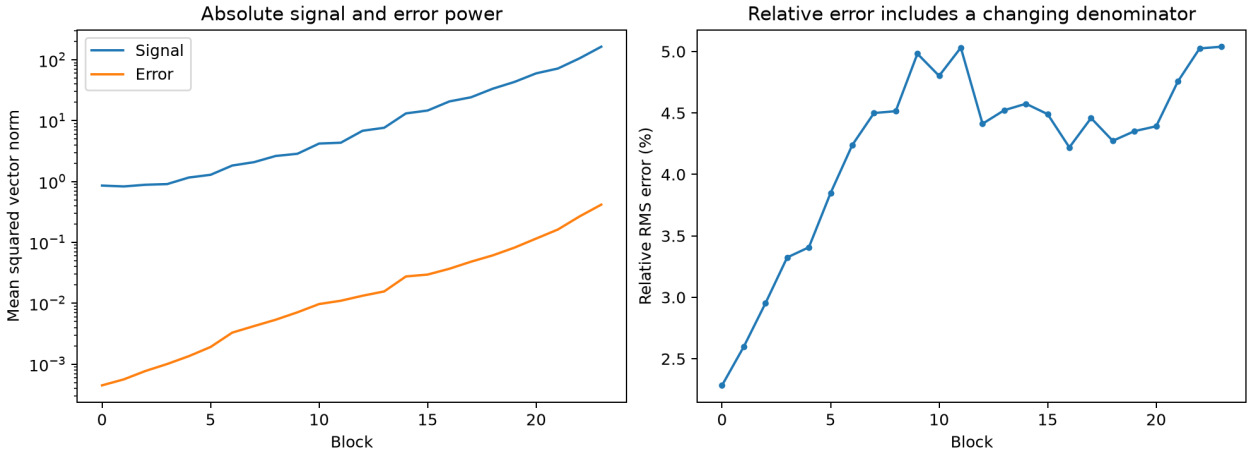


Figure S3: LFM3 absolute error, signal scale, and relative RMS error across blocks. A decline in relative error can occur while absolute error rises; the curves alone do not isolate denoising.

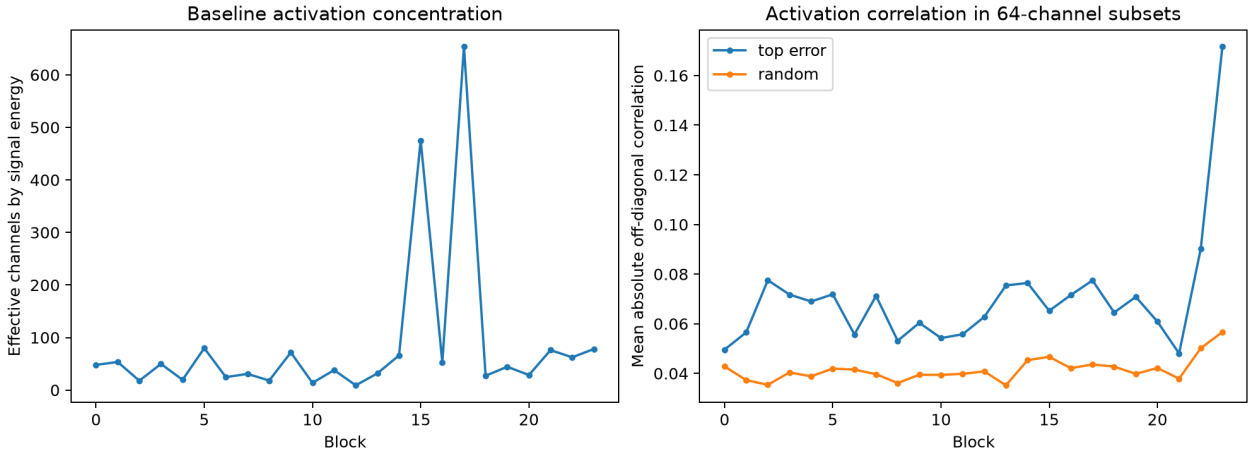


Figure S4: Baseline LFM3 channel structure and error concentration. Correlations of baseline activations in this figure are distinct from the conditional stochastic-error covariance in the main manuscript.

7 Covariance estimator and interpretation

Let $e_{s,t} \in \mathbb{R}^{64}$ denote the output error for seed s at fixed position t, relative to the deterministic baseline, and let $\bar{e}_t$ be its mean across $S = 5$ seeds. The estimator is

$$\widehat{C} = \frac{1}{M} \sum_{t=1}^{M} \frac{1}{S-1} \sum_{s=1}^{S} (e_{s,t} - \bar{e}_t)(e_{s,t} - \bar{e}_t)^{\top}, \qquad M = 8{,}224. \tag{S2}$$

It averages conditional covariance over input positions. It is not obtained by pooling positions and subtracting one global mean. Sufficient statistics include cross sums, the mean outer-product contribution, channel indices, and the resulting matrices. The independent auditor checks all 48 matrices and a direct small-array covariance fixture. All five model draws replay all 8,192 training target losses bitwise before covariance accumulation.

The participation ratio for covariance is $(\operatorname{tr} C)^2 / \operatorname{tr}(C^2)$; the leading-eigenvalue fraction is $\lambda_{\max}(C) / \operatorname{tr} C$. The high-error channels were selected using this same training cohort. Their more concentrated covariance therefore cannot be interpreted as an unbiased population comparison. Averaging conditional covariances may have rank greater than $S - 1$ because input positions have different error subspaces.

8 Additional controls and historical outcomes

8.1 Matched Gaussian noise

Figure S5 presents the training-only, full-parameter GPT-2 comparison. The noise means and per-coordinate variances are matched before FP32/BF16 rounding. All eleven score cases and 740 paired variance entries were audited, and three Gaussian parameter reconstructions independently replayed. The two cohorts use independent random keys. Overlapping intervals do not prove equivalence; they show that these observations do not isolate a special Bernoulli quality benefit.

8.2 Correlation, drift, and calibration

Operator-level moment calculations and simulated calibration probes examine departures from ideal independent sampling; these nonideal models were not executed through native THRML. A

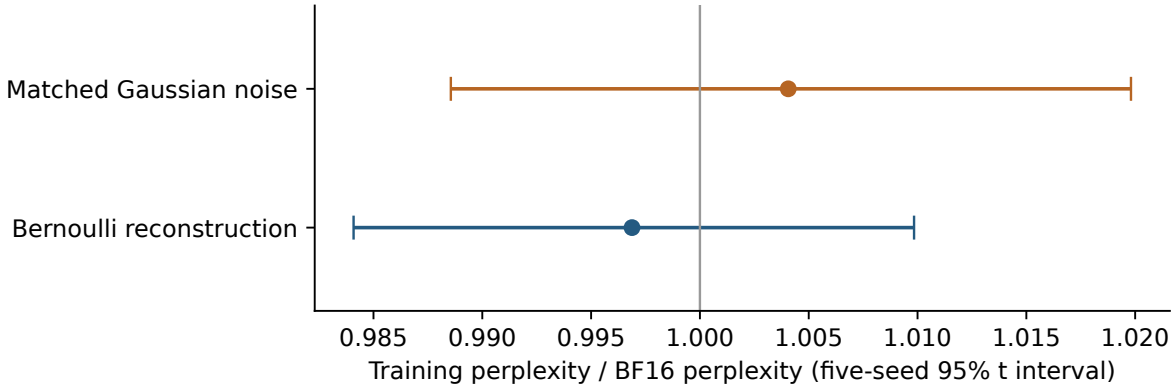


Figure S5: Full-model training diagnostic with matched Gaussian and Bernoulli estimator variance. These are independent five-seed cohorts on fixed training texts, not additional complete held-out results.

stationary binary process with lag correlation ρ^k has finite-sample variance inflation

$$1 + 2\sum_{k=1}^{T-1}(1 - k/T)\rho^k, \tag{S3}$$

which approaches $(1+\rho)/(1-\rho)$ for large T. Accordingly, the common approximation $T_{\text{eff}} \approx T(1-\rho)/(1+\rho)$ has an explicit finite-T limitation. Cross-device correlation adds covariance terms to output variance and must not be represented solely by a scalar effective sample size.

The preserved head investigations include 900 nonideality cases, and finite-probe, prior-aware calibration analyses with 140 cases and 48 optimizer checks. Their calibration priors are assumptions, not measured device populations. An unregularized calibration fit can worsen estimation; an appropriate prior can help in the tested simulator. Probe counts and calibration storage belong in the cost ledger. Drift sensitivity is simulated; no hardware recalibration interval is established.

The full GPT-2 gain/offset experiment uses 16 CPU training cases: one baseline and five seeds each for ideal, mismatched, and oracle-corrected conditions. It audits the matched device arrays and nine parameter replays. Oracle coefficients are known exactly, then re-quantized to the finite control grid. This separates programming bias from sampling variance but does not solve the physical calibration problem.

8.3 Earlier experiments and failures

The earlier adapted LFM3 checkpoint study used 2,048 fixed validation targets and a same-fit BF16 reference of PPL 75.3147. Its T1024 cohort gave PPL 83.9167 and missed the 2% criterion; T4096 gave 76.1024 and also missed the criterion because its upper interval exceeded the margin. T8192 gave 75.1547, ratio 0.99788 with interval [0.98062, 1.01544], and passed. These adapted, shorter-task results are not pooled with the final original-checkpoint evaluation.

An earlier GPT-2 T8192 cohort failed, whereas DistilBERT passed its original criterion. The local range diagnosis in the main paper investigates this variation. An additional 10,240-target LFM3 cloud study was interrupted and has no accepted final quality outcome. A GB10/H100 bridge reproduced stochastic and nominal arrays, while its BF16 reference shifted approximately 0.00515% in perplexity. The initial strict bitwise-control failure is preserved, together with the later documented hardware-specific-reference amendment. These observations are historical context, not additional independent proof of the final Spark claims.

The archive also preserves failed implementation and audit attempts, including calibration bookkeeping errors and revised auditors. Only the corrected completed runs are used for accepted statistics, and exact executed source snapshots remain available. Repeated audits of the same data are not counted as new experimental replications.

9 Resource ledger and storage boundary

Table S2: Exact array payload counts. Commands are the implemented uint16 arrays; ranges are two FP32 values per group. Decoded BF16 weights, conceptual binary states, software objects, and transient buffers are separate.

Configuration	Range groups	Command bytes	Range bytes
GPT-2 uniform	972,186	248,879,616	7,777,488
GPT-2 reduced	978,822	248,879,616	7,830,576
DistilBERT	523,087	133,910,020	4,184,696
LFM3	13,542,448	3,457,603,072	108,339,584

Ideal packed controls are 12 bits per scalar, but the current NumPy implementation stores them in uint16. No packed 12-bit runtime is claimed. Fixed shape, tensor addressing, layout flags, per-tensor sample budgets, and Python bookkeeping incur further overhead. There is no residual side table in the accepted K1 model configurations and no excluded deterministic learned subset. Diagnostic restoration controls explicitly relax that condition.

R1 reconstructs weights once for repeated inference. This preserves a decoded weight array. Discarding the original model after encoding can remove a reference copy, but not the working weights needed by this implementation. Device density, bit storage, free random transitions, all-node observations, and elapsed latency are separate metrics. The original four-element word baseline and the stronger coarse/fractional baseline differ in stored control information; neither licenses a blanket systems-efficiency statement.

10 Evidence, reproduction, and outstanding work

Table S3: Selected accepted evidence identifiers, relative to the dated Spark result directory.

Claim	Artifact identifier
Three complete models	`cross_model_summary_v2/results.json`
GPT-2 saved-output audit	`benchmark_audit_v1/results.json`
DistilBERT saved-output audit	`distilbert_benchmark_audit_v1/results.json`
LFM3 saved-output audit/replay	`lfm3_benchmark_audit_v1/`, `lfm3_benchmark_replay_v1/`
Tied-head intervention	`tied_head_v1/results.json`
LFM3 causal analysis	`lfm3_causal_depth_analysis_v1/results.json`
Covariance audit	`lfm3_error_covariance_v2_audit.json`
Reduced-work audit/replay	`reduced_work_gpu_v1_audit.json`, `reduced_work_replay_v1/`
Strong native control	`native_coarse_fractional_v1_audit.json`
Resource and precision limits	`resource_ledger_final_v1.json`, `precision_limits_v1/`
SDK tests and package hashes	`sdk_build_verification_v2.json`

The package and bounded archive preserve source snapshots indexed by SHA-256, protocols, summaries, independent audits, package versions, and omitted-asset hashes. The manuscript's

`EVIDENCE_MANIFEST.json` maps each bundled figure/input to its originating artifact. Manuscript plots derived from JSON can be regenerated without model weights; raw-output auditors require the omitted model assets and arrays or complete reruns. A hash manifest verifies integrity, not scientific correctness by itself.

For model reproduction, first restore the pinned checkpoints and dataset layouts under their original licenses, recreate the recorded runtime, and use a new output directory. Producers preserve original protocols and typically reject conflicting or already-failed state. Do not edit executed producer files to force old results to pass. Large-model replays, native samplers, and CPU diagnostics use different runtime environments as recorded. Representative module commands are documented in the repository reproduction guide.

The original broad design additionally proposes extensive image-model sweeps, expectation-aware kernels, online adaptive stopping, stochastic training, mixed spatial/temporal optimization, and physical hardware characterization. They are not all established by this manuscript. The final evidence addresses frozen-model feasibility, failure mechanisms, finite-control limits, and selected repairs. General deployment claims would require new untouched tasks, optimized quantization comparisons, long-context/generation evaluations, and measured device programming, retention, correlation, readout, and accumulation costs.